

Arithmetic in Spark Hydro_computeDt

Source: `source/physics/Hydro/HydroMain/Spark/Hydro_computeDt.F90`. This document transcribes the executable arithmetic, including conditional MHD and geometry branches. The routine returns immediately if either `hy_useHydro` or `hy_updateHydroFluxes` is false.

1 Notation and sound speed

Let $q = (i, j, k)$ be an interior cell, $\rho_q = U(\text{DENS}, q)$, $p_q = U(\text{PRES}, q)$, and $\gamma_q = U(\text{GAMC}, q)$. Let $v_{d,q}$ denote the stored fluid velocity in direction d , and $w_{x,i} = \text{uxgrid}(i)$, $w_{y,j} = \text{uygrid}(j)$, $w_{z,k} = \text{uzgrid}(k)$ the grid velocities. The sound speed squared is

$$a_q^2 = \frac{\gamma_q p_q}{\rho_q}. \quad (1)$$

Without `SPARK_GLM`, each directional propagation speed is

$$c_{f,x,q}^2 = c_{f,y,q}^2 = c_{f,z,q}^2 = a_q^2. \quad (2)$$

2 GLM-MHD fast magneto-acoustic speeds

When `SPARK_GLM` is defined, let $B_{d,q} = U(\text{MAG}d, q)$. The code computes

$$b_{x,q}^2 = \frac{B_{x,q}^2}{\rho_q}, \quad b_{y,q}^2 = \frac{B_{y,q}^2}{\rho_q}, \quad b_{z,q}^2 = \frac{B_{z,q}^2}{\rho_q}, \quad (3)$$

$$b_q^2 = b_{x,q}^2 + b_{y,q}^2 + b_{z,q}^2. \quad (4)$$

For each $d \in \{x, y, z\}$,

$$c_{f,d,q}^2 = \frac{1}{2} \left[a_q^2 + b_q^2 + \sqrt{(a_q^2 + b_q^2)^2 - 4a_q^2 b_{d,q}^2} \right]. \quad (5)$$

The routine uses these expressions directly, with no additional magnetic normalization factor or square-root argument clamp.

3 Inverse cell lengths and geometry

Let i_0, j_0, k_0 be the lower interior indices and define

$$h_x = \text{dx}(i_0), \quad h_y = \text{dy}(j_0), \quad h_z = \text{dz}(k_0). \quad (6)$$

The initial inverse lengths are

$$\lambda_x = \frac{1}{h_x}, \quad \lambda_y = \frac{1}{h_y} \quad (N > 1), \quad \lambda_z = \frac{1}{h_z} \quad (N > 2), \quad (7)$$

where $N = \text{NDIM}$. These initial values are reused throughout the interior-cell loop except for the following geometry overrides.

For cylindrical geometry in three dimensions, $x(i) = r_i$ and the z coordinate represents ϕ :

$$\lambda_{z,q} = \frac{1}{r_i \, \text{dz}(k)}. \quad (8)$$

The x and y inverse lengths retain their initial values.

For spherical geometry when $N > 1$, $x(i) = r_i$, $y(j) = \theta_j$, and z represents ϕ :

$$\lambda_{y,q} = \frac{1}{r_i \, \text{dy}(j)}, \quad \lambda_{z,q} = \frac{1}{r_i \sin \theta_j \, \text{dz}(k)}. \quad (9)$$

The source assigns both spherical expressions for $N > 1$, but uses the z rate in the timestep calculation only for $N = 3$. Thus the Cartesian spacings, and any directions without a geometry override, are taken from the lower interior cell rather than recomputed from each cell's spacing.

4 Directional rates and accumulated wave speed

Define the speed relative to the grid, including signal propagation, by

$$s_{x,q} = |v_{x,q} - w_{x,i}| + \sqrt{c_{f,x,q}^2}, \quad (10)$$

$$s_{y,q} = |v_{y,q} - w_{y,j}| + \sqrt{c_{f,y,q}^2}, \quad (11)$$

$$s_{z,q} = |v_{z,q} - w_{z,k}| + \sqrt{c_{f,z,q}^2}. \quad (12)$$

The cell rate (the variable `dt_ltemp`, with units of inverse time) is

$$R_q = \begin{cases} s_{x,q} \lambda_x, & N = 1, \\ \max(s_{x,q} \lambda_x, s_{y,q} \lambda_{y,q}), & N = 2, \\ \max(s_{x,q} \lambda_x, s_{y,q} \lambda_{y,q}, s_{z,q} \lambda_{z,q}), & N = 3. \end{cases} \quad (13)$$

Let $\mathcal{I}$ be the set of interior cells and D_N the active directions. The stored maximum hyperbolic speed is updated as

$$\text{hy_lChyp}_{\text{out}} = \max \left(\text{hy_lChyp}_{\text{in}}, \max_{q \in \mathcal{I}} \max_{d \in D_N} s_{d,q} \right). \quad (14)$$

This routine does not reset `hy_lChyp` before accumulating speeds.

5 CFL timestep and limiting location

Let $C = \text{hy_cfl}$. The exact loop starts with $R_* = 0$ and a five-component location vector $\ell_* = (0, 0, 0, 0, 0)$. For each cell,

$$\text{if } CR_q > CR_* : \quad R_* \leftarrow R_q, \quad \ell_* \leftarrow (i, j, k, \text{blockDesc\%level}, \text{hy_meshMe}). \quad (15)$$

The loop visits i fastest, then j , then k . Its strict comparison preserves the first location attaining a positive maximum when $C > 0$. Under the usual positive-CFL and nonnegative-rate assumptions,

$$R_* = \max \left(0, \max_{q \in \mathcal{I}} R_q \right). \quad (16)$$

After the loop, the candidate timestep is

$$\Delta t_{\text{candidate}} = \begin{cases} C/R_*, & R_* \neq 0, \\ \text{huge}(1.0), & R_* = 0. \end{cases} \quad (17)$$

Here `huge(1.0)` is the largest finite representable value of the Fortran real kind of the literal, rather than mathematical infinity. For positive rates this is equivalently

$$\Delta t_{\text{candidate}} = C \min_{q \in \mathcal{I}} \min_{d \in D_N} \frac{1}{s_{d,q} \lambda_{d,q}}. \quad (18)$$

The incoming constraint is reduced by

$$\text{dtCheck}_{\text{out}} = \min(\text{dtCheck}_{\text{in}}, \Delta t_{\text{candidate}}). \quad (19)$$

Only if $\Delta t_{\text{candidate}} < \text{dtCheck}_{\text{in}}$ does `dtMinLoc` become ℓ_* ; otherwise its incoming value is retained. The optional argument `extraInfo` is not assigned. The local `tempCfl` always equals C ; `dtCflLoc` is assigned C but is not subsequently used.